🛡️ Network Intrusion Detection — CIC-IDS-2017

Traditional Machine Learning Pipeline

Dataset: CIC-IDS-2017 — Canadian Institute for Cybersecurity
Goal: Phân loại network traffic thành **Benign** hoặc các loại **Attack** (DoS, DDoS, PortScan, Brute Force, Bot, v.v.)

Pipeline:

1. 📥 Download dataset từ Kaggle (tự động)
2. 🧹 Làm sạch & tiền xử lý
3. 📊 Xử lý class imbalance (Combined: UnderSampling + SMOTE)
4. 🤖 Train: Random Forest, XGBoost, Logistic Regression
5. 📈 Đánh giá: Confusion Matrix, ROC Curves, Feature Importance

📦 1. Cài đặt thư viện

```
%%capture
!pip install imbalanced-learn xgboost kaggle -q
```

```
Start coding or generate with AI.
```

📥 2. Tải Dataset

Cách 1 (Khuyên dùng): Kaggle API — Tự động ✅

Dataset CIC-IDS-2017 có sẵn trên Kaggle. Bạn cần **Kaggle API key** (`kaggle.json`).

Lấy API key: Kaggle → Account → Create New API Token → tải về `kaggle.json`

```
import os, json
from google.colab import files

# — Upload kaggle.json —
print("Upload file kaggle.json của bạn:")
uploaded = files.upload()

# Cài đặt credentials
os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "wb") as f:
    f.write(uploaded[list(uploaded.keys())[0]])
os.chmod("/root/.kaggle/kaggle.json", 0o600)
print("✅ Kaggle credentials đã được cài đặt!")
```

```
Upload file kaggle.json của bạn:
Choose Files kaggle.json
kaggle.json(application/json) - 68 bytes, last modified: 4/24/2026 - 100% done
Saving kaggle.json to kaggle (2).json
✅ Kaggle credentials đã được cài đặt!
```

```
!kaggle datasets download chethuhn/network-intrusion-dataset -p /content/cicids2017 --unzip
```

```
Dataset URL: https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset
License(s): CC0-1.0
Downloading network-intrusion-dataset.zip to /content/cicids2017
100% 230M/230M [00:02<00:00, 117MB/s]
```

Cách 2: Upload thủ công từ máy tính 💻

Nếu bạn đã tải dataset về máy (từ trang chính thức UNB), dùng cell bên dưới.

> **Lưu ý:** File gốc từ UNB thường nằm trong thư mục `MachineLearningCVE/`

```
import glob, os
csv_files = glob.glob("/content/cicids2017/*.csv", recursive=True)
print("CSV files tìm thấy:")
for f in sorted(csv_files):
    print(f"  {f}  ({os.path.getsize(f)/1e6:.1f} MB)")
```

```
CSV files tìm thấy:
  /content/cicids2017/Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv  (77.1 MB)
  /content/cicids2017/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv  (76.9 MB)
  /content/cicids2017/Friday-WorkingHours-Morning.pcap_ISCX.csv  (58.3 MB)
  /content/cicids2017/Monday-WorkingHours.pcap_ISCX.csv  (176.9 MB)
  /content/cicids2017/Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv  (83.1 MB)
  /content/cicids2017/Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv  (52.0 MB)
  /content/cicids2017/Tuesday-WorkingHours.pcap_ISCX.csv  (135.1 MB)
  /content/cicids2017/Wednesday-workingHours.pcap_ISCX.csv  (225.2 MB)
```

Cách 3: Google Drive 📂

Nếu dataset đã lưu trên Google Drive của bạn.

```
DATA_DIR = "/content/cicids2017"
```

⚙️ 3. Import thư viện & Cấu hình

```
import warnings, glob, os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, roc_curve, f1_score
)
from sklearn.preprocessing import label_binarize
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from xgboost import XGBClassifier
from sklearn.tree import DecisionTreeClassifier
from lightgbm import LGBMClassifier

warnings.filterwarnings("ignore")
np.random.seed(42)
sns.set_theme(style="whitegrid")

# ──────────────── CẤU HÌNH ────────────────
CONFIG = {
    "max_samples_per_class": 50_000,  # Giới hạn mỗi class (tránh OOM)
    "rare_class_threshold":  500,     # Class < 500 mẫu → gộp vào Rare_Attack
    "test_size":             0.2,
    "output_dir":            "/content/ids_results/",
}
os.makedirs(CONFIG["output_dir"], exist_ok=True)
print("✅ Cấu hình xong!")
print(f"   max_samples_per_class : {CONFIG['max_samples_per_class']:,}")
print(f"   rare_class_threshold  : {CONFIG['rare_class_threshold']}")
```

```
✅ Cấu hình xong!
   max_samples_per_class : 50,000
   rare_class_threshold  : 500
```

📂 4. Load & Khám phá Dataset

```
def load_cicids2017(data_dir: str) -> pd.DataFrame:
    csv_files = glob.glob(os.path.join(data_dir, "**/*.csv"), recursive=True)
    if not csv_files:
        csv_files = glob.glob(os.path.join(data_dir, "*.csv"))
    if not csv_files:
        raise FileNotFoundError(f"Không tìm thấy CSV trong {data_dir}\n"
                                "Hãy chạy lại cell tải dataset ở trên!")
    print(f"📁 Load {len(csv_files)} file CSV...")
    dfs = []
    for f in sorted(csv_files):
        df_tmp = pd.read_csv(f, low_memory=False)
        print(f"  ✔ {os.path.basename(f):50s} → {len(df_tmp):>8,} rows")
        dfs.append(df_tmp)
    df = pd.concat(dfs, ignore_index=True)
    print(f"\n📊 Tổng: {df.shape[0]:,} rows × {df.shape[1]} columns")
    return df

df_raw = load_cicids2017(DATA_DIR)
```

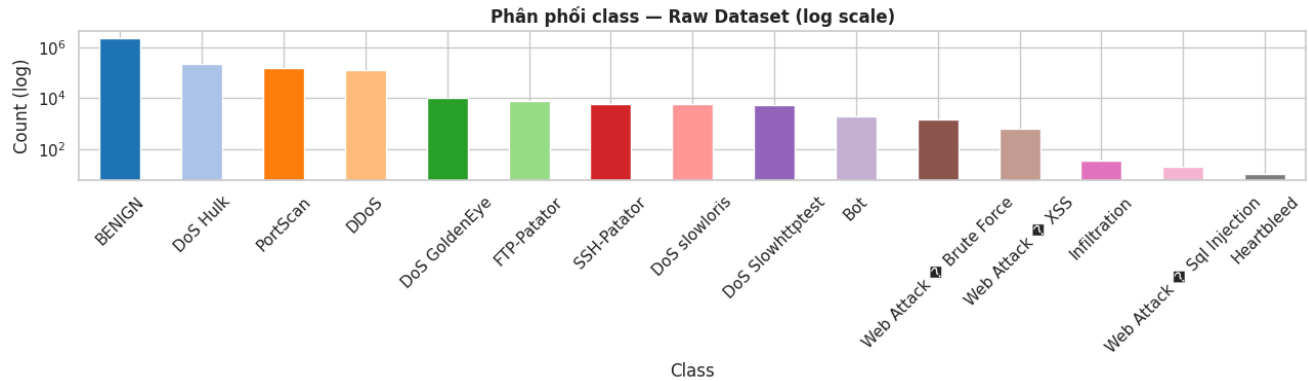
```
📁 Load 8 file CSV...
  ✔ Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv   →  225,745 rows
  ✔ Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv →  286,467 rows
  ✔ Friday-WorkingHours-Morning.pcap_ISCX.csv          →  191,033 rows
  ✔ Monday-WorkingHours.pcap_ISCX.csv                  →  529,918 rows
  ✔ Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv →  288,602 rows
  ✔ Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv →  170,366 rows
  ✔ Tuesday-WorkingHours.pcap_ISCX.csv                 →  445,909 rows
  ✔ Wednesday-workingHours.pcap_ISCX.csv               →  692,703 rows

📊 Tổng: 2,830,743 rows × 79 columns
```

```
# Xem cấu trúc dataset
print("=== Thông tin dataset ===")
print(f"Shape: {df_raw.shape}")
print(f"\nCác cột đầu tiên:")
print(df_raw.head(3).to_string())
```

```
=== Thông tin dataset ===
Shape: (2830743, 79)

Các cột đầu tiên:
    Destination Port   Flow Duration   Total Fwd Packets   Total Backward Packets  Total Length of Fwd Packets   Tota
0              54865               3                   2                         0                            12
1              55054             109                   1                         1                             6
2              55055              52                   1                         1                             6
```

```
# Tìm cột Label
df_raw.columns = df_raw.columns.str.strip()

label_candidates = ["Label", "label", "Attack", " Label"]
label_col = None
for c in label_candidates:
    if c.strip() in df_raw.columns:
        label_col = c.strip()
        break

print(f"✅ Cột label: '{label_col}'")
print(f"\n📊 Phân phối class (RAW):")
vc = df_raw[label_col].value_counts()
print(vc.to_string())

fig, ax = plt.subplots(figsize=(13, 4))
vc.plot(kind="bar", ax=ax, color=plt.cm.tab20.colors[:len(vc)], edgecolor="white")
ax.set_title("Phân phối class — Raw Dataset (log scale)", fontweight="bold")
ax.set_yscale("log")
ax.set_xlabel("Class"); ax.set_ylabel("Count (log)")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.show()
```

```
✅ Cột label: 'Label'

📊 Phân phối class (RAW):
Label
BENIGN                        2273097
DoS Hulk                       231073
PortScan                       158930
DDoS                           128027
DoS GoldenEye                   10293
FTP-Patator                      7938
SSH-Patator                      5897
DoS slowloris                    5796
DoS Slowhttptest                 5499
Bot                              1966
Web Attack – Brute Force         1507
Web Attack – XSS                  652
Infiltration                       36
Web Attack – Sql Injection         21
Heartbleed                         11
```



🧹 5. Làm sạch dữ liệu

```
df = df_raw.copy()
df[label_col] = df[label_col].str.strip()

# Xử lý Infinity & NaN
df.replace([np.inf, -np.inf], np.nan, inplace=True)
n_nan = df.isnull().sum().sum()
df.dropna(inplace=True)
print(f"🗑️  Đã xóa {n_nan:,} giá trị Infinity/NaN")
print(f"📊 Còn lại: {df.shape[0]:,} rows")

# Kiểm tra duplicate
n_dup = df.duplicated().sum()
df.drop_duplicates(inplace=True)
print(f"🗑️  Đã xóa {n_dup:,} dòng trùng lặp")
print(f"📊 Final: {df.shape[0]:,} rows × {df.shape[1]} cols")
```

```
🗑️  Đã xóa 5,734 giá trị Infinity/NaN
📊 Còn lại: 2,827,876 rows
🗑️  Đã xóa 307,078 dòng trùng lặp
📊 Final: 2,520,798 rows × 79 cols
```

✂️ 6. Stratified Sampling & Xử lý Class Hiếm

Vấn đề với CIC-IDS-2017:

- Dataset gốc: ~2.8 triệu dòng → có thể gây OOM trên Colab
- Class rất mất cân bằng: BENIGN chiếm ~80%, một số attack chỉ có 11 mẫu

Giải pháp:

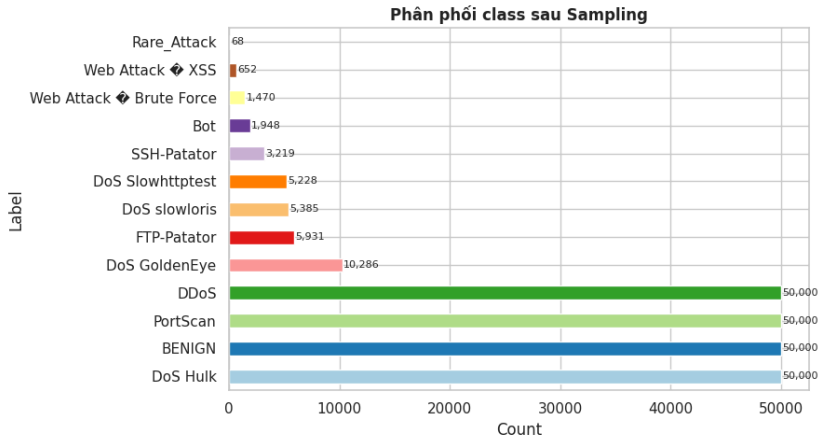
- Gộp các class < 500 mẫu vào `Rare_Attack`

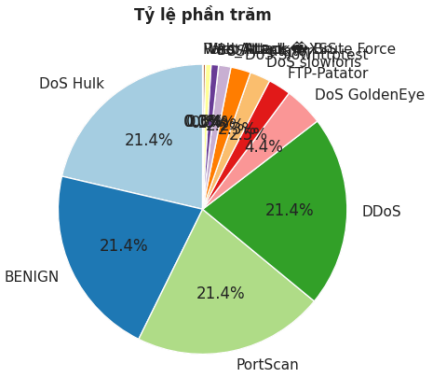
- Giới hạn mỗi class tối đa 50,000 mẫu

```
def sample_dataset(df, label_col, config):
    max_per = config["max_samples_per_class"]
    rare_th = config["rare_class_threshold"]

    # Gộp class hiếm
    vc = df[label_col].value_counts()
    rare = vc[vc < rare_th].index.tolist()
    if rare:
        print(f"⚠️  Gộp {len(rare)} class hiếm vào 'Rare_Attack':")
        for r in rare:
            print(f"    - {r}: {vc[r]} mẫu")
        df = df.copy()
        df[label_col] = df[label_col].apply(lambda x: "Rare_Attack" if x in rare else x)

    # Stratified sampling
    print(f"\n📏 Sampling (tối đa {max_per:,} mẫu/class):")
    parts = []
    for cls, grp in df.groupby(label_col):
        if len(grp) > max_per:
            parts.append(grp.sample(n=max_per, random_state=42))
            print(f"  {cls:40s}: {len(grp):>8,} → {max_per:,} ✂️")
        else:
            parts.append(grp)
            print(f"  {cls:40s}: {len(grp):>8,} (kept)")

    df_s = pd.concat(parts).sample(frac=1, random_state=42).reset_index(drop=True)
    print(f"\n✅ Sau sampling: {df_s.shape[0]:,} rows")
    return df_s

df = sample_dataset(df, label_col, CONFIG)
```

```
⚠️  Gộp 3 class hiếm vào 'Rare_Attack':
    - Infiltration: 36 mẫu
    - Web Attack ▯ Sql Injection: 21 mẫu
    - Heartbleed: 11 mẫu

📏 Sampling (tối đa 50,000 mẫu/class):
  BENIGN                                   : 2,095,057 → 50,000 ✂️
  Bot                                      :     1,948 (kept)
  DDoS                                     :   128,014 → 50,000 ✂️
  DoS GoldenEye                            :    10,286 (kept)
  DoS Hulk                                 :   172,846 → 50,000 ✂️
  DoS Slowhttptest                         :     5,228 (kept)
  DoS slowloris                            :     5,385 (kept)
  FTP-Patator                              :     5,931 (kept)
  PortScan                                 :    90,694 → 50,000 ✂️
  Rare_Attack                              :        68 (kept)
  SSH-Patator                              :     3,219 (kept)
  Web Attack ▯ Brute Force                 :     1,470 (kept)
  Web Attack ▯ XSS                         :       652 (kept)

✅ Sau sampling: 234,187 rows
```

```
# Visualize sau sampling
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Count
vc = df[label_col].value_counts()
vc.plot(kind="barh", ax=axes[0], color=plt.cm.Paired.colors[:len(vc)])
axes[0].set_title("Phân phối class sau Sampling", fontweight="bold")
axes[0].set_xlabel("Count")
for i, (v, name) in enumerate(zip(vc.values, vc.index)):
    axes[0].text(v + 100, i, f"{v:,}", va="center", fontsize=8)

# Pie
axes[1].pie(vc.values, labels=vc.index, autopct="%1.1f%%",
            colors=plt.cm.Paired.colors[:len(vc)], startangle=90)
axes[1].set_title("Tỷ lệ phần trăm", fontweight="bold")

plt.tight_layout(); plt.show()
print(f"\nTổng mẫu: {len(df):,}")
```






```
Tổng mẫu: 234,187
```

🔧 7. Feature Engineering & Chuẩn bị dữ liệu

```
# Drop các cột không phải feature
drop_cols = [label_col, "Flow ID", "Source IP", "Destination IP",
             "Timestamp", "Src IP", "Dst IP"]
drop_cols = [c for c in drop_cols if c in df.columns]

X = df.drop(columns=drop_cols).select_dtypes(include=[np.number])
y_raw = df[label_col]

# Label encoding
le = LabelEncoder()
y = le.fit_transform(y_raw)

print(f"✅ Feature matrix: {X.shape}")
print(f"✅ Classes ({len(le.classes_)}):")
for i, cls in enumerate(le.classes_):
    print(f"  [{i}] {cls}")
```

```
✅ Feature matrix: (234187, 78)
✅ Classes (13):
  [0] BENIGN
  [1] Bot
  [2] DDoS
  [3] DoS GoldenEye
  [4] DoS Hulk
  [5] DoS Slowhttptest
  [6] DoS slowloris
  [7] FTP-Patator
  [8] PortScan
  [9] Rare_Attack
  [10] SSH-Patator
  [11] Web Attack  Brute Force
  [12] Web Attack  XSS
```

```
# Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=CONFIG["test_size"], stratify=y, random_state=42
)

# Scaling
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)

print(f"📊 Train set: {X_train_s.shape}")
```

```
print(f"📊 Test set : {X_test_s.shape}")

# Kiểm tra phân phối train
train_counts = pd.Series(y_train).value_counts().sort_index()
print("\nPhân phối Train (trước SMOTE):")
for i, cnt in train_counts.items():
    print(f"  [{i}] {le.classes_[i]:40s}: {cnt:,}")
```

```
📊 Train set: (187349, 78)
📊 Test set : (46838, 78)

Phân phối Train (trước SMOTE):
  [0] BENIGN                                  : 40,000
  [1] Bot                                     : 1,558
  [2] DDoS                                    : 40,000
  [3] DoS GoldenEye                           : 8,229
  [4] DoS Hulk                                : 40,000
  [5] DoS Slowhttptest                        : 4,182
  [6] DoS slowloris                           : 4,308
  [7] FTP-Patator                             : 4,745
  [8] PortScan                                : 40,000
  [9] Rare_Attack                             : 54
  [10] SSH-Patator                             : 2,575
  [11] Web Attack – Brute Force                : 1,176
  [12] Web Attack – XSS                        : 522
```

⚖️ 8. Xử lý Class Imbalance

Chiến lược Combined:

1. **RandomUnderSampler** → Cắt class BENIGN xuống 3× class lớn thứ 2
2. **SMOTE** → Oversample các class minority bằng synthetic samples

Nguyên tắc xác định số lượng mẫu SMOTE cho mỗi nhãn

1. Tỷ lệ mất cân bằng (Imbalance Ratio — IR)

$$IR = \frac{N_{\text{majority}}}{N_{\text{minority}}}$$

Khoảng IR	Mức độ mất cân bằng
$IR < 10$	Nhẹ — SMOTE ít cần thiết
$10 \le IR \le 100$	Trung bình — áp dụng SMOTE có kiểm soát
$IR > 100$	Nghiêm trọng — nguy cơ tỷ lệ synthetic quá cao

---

2. Công thức căn bậc hai (Square Root Target)

$$N_{\text{target}} = \sqrt{N_{\text{majority}} \times N_{\text{minority}}}$$

Trung bình nhân đặt target nằm giữa hai cực — không quá thấp (bỏ sót thông tin) và không quá cao (thổi phồng dữ liệu giả).

Ví dụ:

$$N_{\text{target}} = \sqrt{40{,}000 \times 1{,}558} = \sqrt{62{,}320{,}000} \approx 7{,}894$$

---

3. Ngưỡng an toàn — Quy tắc $10\times$

$$N_{\text{target}} \le \min\left(10 \times N_{\text{minority}},\ N_{\text{majority}}\right)$$

Nếu SMOTE tạo ra hơn $10\times$ số mẫu thực, tỷ lệ synthetic vượt quá 90% — model học chủ yếu từ dữ liệu giả, kết quả không đáng tin cậy.

---

4. Công thức kết hợp (Khuyến nghị)

$$\boxed{N_{\text{target}} = \min\left(\sqrt{N_{\text{maj}} \times N_{\text{min}}},\quad 10 \times N_{\text{min}},\quad N_{\text{maj}}\right)}$$

---

5. Tỷ lệ mẫu synthetic

$$\%_{\text{synthetic}} = \frac{N_{\text{target}} - N_{\text{real}}}{N_{\text{target}}} \times 100\%$$

$\%_{\text{synthetic}}$	Độ tin cậy	Diễn giải
$< 50\%$	🟢 Cao	Đáng tin cậy
$50\% - 80\%$	🟡 Trung bình	Chấp nhận được, nên ghi chú khi báo cáo
$> 80\%$	🔴 Thấp	Kết quả bị thổi phồng — không nên kết luận

---

6. Áp dụng vào CIC-IDS-2017

Đặt $N_{\text{maj}} = 40{,}000$ (lớp BENIGN sau undersampling). Target cho mỗi lớp:

$$N_{\text{target},\, i} = \min\left(\sqrt{40{,}000 \times N_i},\ 10 \times N_i,\ 40{,}000\right)$$

Nhãn	$N_{\text{thực}}$	$\sqrt{N_{\text{maj}} \cdot N}$	$10 \times N$	Target
DoS GoldenEye	8,229	18,135	82,290	**18,135**
DoS Slowhttptest	4,182	12,930	41,820	**12,930**
DoS slowloris	4,308	13,127	43,080	**13,127**
FTP-Patator	4,745	13,775	47,450	**13,775**
SSH-Patator	2,575	10,149	25,750	**10,149**
Bot	1,558	7,894	15,580	**7,894**
Web Attack - Brute Force	1,176	6,861	11,760	**6,861**
Web Attack - XSS	522	4,569	5,220	**4,569**
Rare_Attack	54	1,470	540	**540** ← bị cap bởi $10\times$

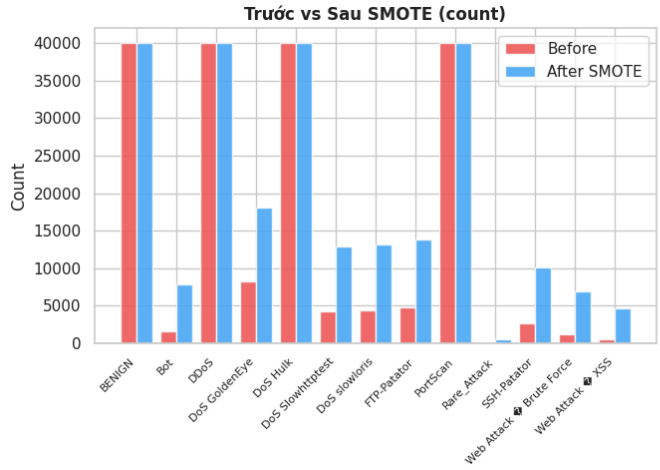
> ⚠️ **Lưu ý:** Rare_Attack bị giới hạn bởi quy tắc $10\times$ ($540 < 1{,}470$), tránh tình trạng 54 mẫu thực bị thổi phồng quá mức.

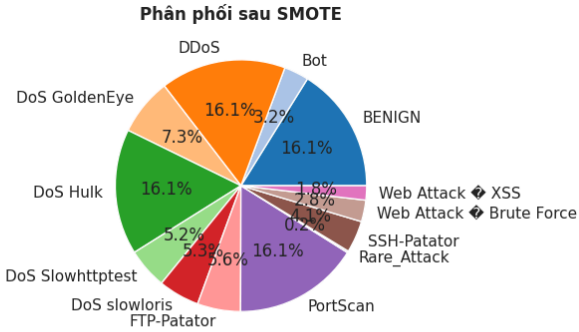
```
def compute_smote_targets(y_train, le, majority_cap=40_000):
    counts = np.bincount(y_train)
    majority_cls = int(np.argmax(counts))
    N_maj = counts[majority_cls]

    strategy = {}
    print(f"{'Class':45s} {'N_real':>8} {'sqrt_target':>12} {'10x_cap':>10} {'Final':>8}")
    print("—" * 90)

    for i, cnt in enumerate(counts):
        if i == majority_cls:
            print(f"  {le.classes_[i]:43s} {cnt:>8,}   [majority — undersample]")
            continue

        sqrt_target = int(np.sqrt(N_maj * cnt))
        cap_10x     = int(cnt * 10)
        target      = min(sqrt_target, cap_10x, majority_cap)

        print(f"  {le.classes_[i]:43s} {cnt:>8,} {sqrt_target:>12,} {cap_10x:>10,} {target:>8,}")

        if target > cnt:
            strategy[i] = target

    return strategy

sampling_strategy = compute_smote_targets(y_train, le)

def handle_imbalance_combined(X_tr, y_tr):
    counts = np.bincount(y_tr)
    second_largest = sorted(counts)[-2]
    target_majority = min(counts.max(), second_largest * 3)
    majority_class  = int(np.argmax(counts))

    print(f"🔽 UnderSampling class [{majority_class}] {le.classes_[majority_class]}: "
          f"{counts[majority_class]:,} → {target_majority:,}")

    under_strategy = {}
    if counts[majority_class] > target_majority:
        under_strategy[majority_class] = int(target_majority)
        under = RandomUnderSampler(sampling_strategy=under_strategy, random_state=42)
        X_tr, y_tr = under.fit_resample(X_tr, y_tr)

    # SMOTE
    min_samples = int(np.bincount(y_tr).min())
```

```
    k = min(5, min_samples - 1)
    print(f"🔼 SMOTE với k_neighbors={k}...")
    smote = SMOTE(sampling_strategy=sampling_strategy, random_state=42, k_neighbors=5)
    X_res, y_res = smote.fit_resample(X_train_s, y_train)

    print("\n✅ Sau resampling:")
    for cls_i, cnt in zip(*np.unique(y_res, return_counts=True)):
        print(f"  [{cls_i}] {le.classes_[cls_i]:40s}: {cnt:,}")
    return X_res, y_res

X_train_res, y_train_res = handle_imbalance_combined(X_train_s, y_train)
```

```
Class                                          N_real  sqrt_target    10x_cap    Final
──────────────────────────────────────────────────────────────────────────────────────
  BENIGN                                       40,000   [majority — undersample]
  Bot                                           1,558        7,894     15,580    7,894
  DDoS                                         40,000       40,000    400,000   40,000
  DoS GoldenEye                                 8,229       18,142     82,290   18,142
  DoS Hulk                                     40,000       40,000    400,000   40,000
  DoS Slowhttptest                              4,182       12,933     41,820   12,933
  DoS slowloris                                 4,308       13,127     43,080   13,127
  FTP-Patator                                   4,745       13,776     47,450   13,776
  PortScan                                     40,000       40,000    400,000   40,000
  Rare_Attack                                      54        1,469        540      540
  SSH-Patator                                   2,575       10,148     25,750   10,148
  Web Attack ⓪ Brute Force                      1,176        6,858     11,760    6,858
  Web Attack ⓪ XSS                                522        4,569      5,220    4,569
🔽 UnderSampling class [0] BENIGN: 40,000 → 40,000
🔼 SMOTE với k_neighbors=5...

✅ Sau resampling:
  [0] BENIGN                                  : 40,000
  [1] Bot                                     : 7,894
  [2] DDoS                                    : 40,000
  [3] DoS GoldenEye                           : 18,142
  [4] DoS Hulk                                : 40,000
  [5] DoS Slowhttptest                        : 12,933
  [6] DoS slowloris                           : 13,127
  [7] FTP-Patator                             : 13,776
  [8] PortScan                                : 40,000
  [9] Rare_Attack                             : 540
  [10] SSH-Patator                             : 10,148
  [11] Web Attack ⓪ Brute Force                : 6,858
  [12] Web Attack ⓪ XSS                        : 4,569
```

```
# So sánh trước/sau SMOTE
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

before = pd.Series(y_train).value_counts().sort_index()
after  = pd.Series(y_train_res).value_counts().sort_index()
labels = [le.classes_[i] for i in before.index]

x = np.arange(len(labels))
w = 0.4
axes[0].bar(x - w/2, before.values, width=w, label="Before", color="#EF5350", alpha=0.85)
axes[0].bar(x + w/2, [after.get(i, 0) for i in before.index], width=w, label="After SMOTE", color="#42A5F5", alpha=0
axes[0].set_xticks(x); axes[0].set_xticklabels(labels, rotation=45, ha="right", fontsize=8)
axes[0].set_title("Trước vs Sau SMOTE (count)", fontweight="bold")
axes[0].legend(); axes[0].set_ylabel("Count")

axes[1].pie(after.values, labels=[le.classes_[i] for i in after.index],
            autopct="%1.1f%%", colors=plt.cm.tab20.colors[:len(after)])
axes[1].set_title("Phân phối sau SMOTE", fontweight="bold")

plt.tight_layout(); plt.show()
```





🤖 9. Huấn luyện Models

Trước khi SMOTE

```
from sklearn.metrics import (
    accuracy_score, balanced_accuracy_score,
    matthews_corrcoef, f1_score, roc_auc_score
)

def evaluate(model, name):
    y_pred = model.predict(X_test_s)
    y_prob = model.predict_proba(X_test_s)

    print(classification_report(y_test, y_pred,
          target_names=le.classes_, zero_division=0))

    metrics = {
        "model":             name,
        # ── Metric chính ──
        "f1_macro":          f1_score(y_test, y_pred, average="macro",    zero_division=0),
        "f1_weighted":       f1_score(y_test, y_pred, average="weighted", zero_division=0),
        "roc_auc":           roc_auc_score(y_test, y_prob, multi_class="ovr", average="macro"),
        # ── Thêm vào ──
        "balanced_accuracy": balanced_accuracy_score(y_test, y_pred),  # accuracy trung bình mỗi class
        "mcc":               matthews_corrcoef(y_test, y_pred),         # tổng hợp cả 4 ô confusion matrix
        # ── Giữ để debug ──
        "accuracy":          accuracy_score(y_test, y_pred),            # chỉ tham khảo
        "y_pred":            y_pred,
        "y_prob":            y_prob,
    }

    print(f"  F1 Macro           : {metrics['f1_macro']:.4f}   ← quan trọng nhất (imbalanced)")
    print(f"  F1 Weighted        : {metrics['f1_weighted']:.4f}")
    print(f"  ROC-AUC (macro)    : {metrics['roc_auc']:.4f}   ← độc lập threshold")
    print(f"  Balanced Accuracy  : {metrics['balanced_accuracy']:.4f}   ← accuracy công bằng mỗi class")
    print(f"  MCC                : {metrics['mcc']:.4f}   ← [-1, 1], tin cậy nhất khi imbalanced")
    print(f"  Accuracy (ref only): {metrics['accuracy']:.4f}   ← không dùng để so sánh model")

    return metrics
```

```
models = {
    "DecisionTree": DecisionTreeClassifier(
        max_depth=15,
        min_samples_split=5,
        min_samples_leaf=2,
        class_weight="balanced",
```

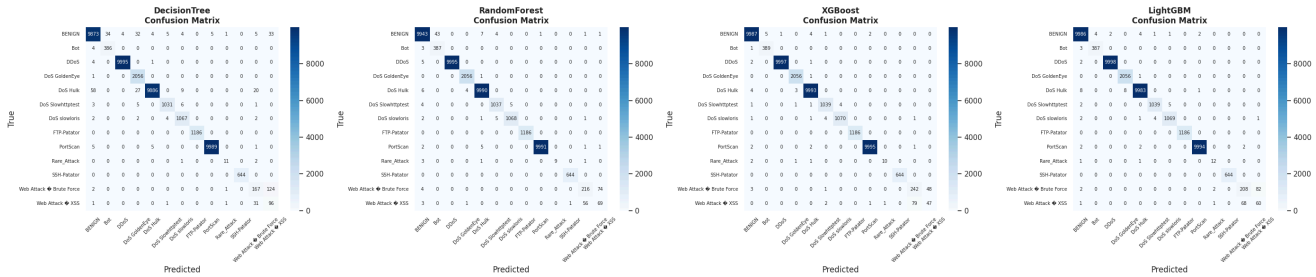
```
        random_state=42
    ),
    "RandomForest": RandomForestClassifier(
        n_estimators=200, max_depth=20, min_samples_split=5,
        class_weight="balanced", n_jobs=-1, random_state=42
    ),
    "XGBoost": XGBClassifier(
        n_estimators=200, max_depth=8, learning_rate=0.1,
        subsample=0.8, colsample_bytree=0.8,
        eval_metric="mlogloss", n_jobs=-1, random_state=42,
        verbosity=0
    ),
    "LightGBM": LGBMClassifier(
        n_estimators=300, max_depth=8, learning_rate=0.05,
        num_leaves=63, subsample=0.8, colsample_bytree=0.8,
        class_weight="balanced", n_jobs=-1, random_state=42,
        verbose=-1
    ),
}

trained = {}
import time

for name, model in models.items():
    print(f"\n{'-'*50}")
    print(f"🏋️  Training {name}...")
    t0 = time.time()
    model.fit(X_train_s, y_train)
    elapsed = time.time() - t0
    trained[name] = model
    print(f"✅ Done in {elapsed:.1f}s")
```

```
--------------------------------------------------
🏋️  Training DecisionTree...
✅ Done in 6.7s

--------------------------------------------------
🏋️  Training RandomForest...
✅ Done in 62.2s

--------------------------------------------------
🏋️  Training XGBoost...
✅ Done in 111.9s

--------------------------------------------------
🏋️  Training LightGBM...
✅ Done in 148.9s
```

```
results = {}
for name, model in trained.items():
    results[name] = evaluate(model, name)
```

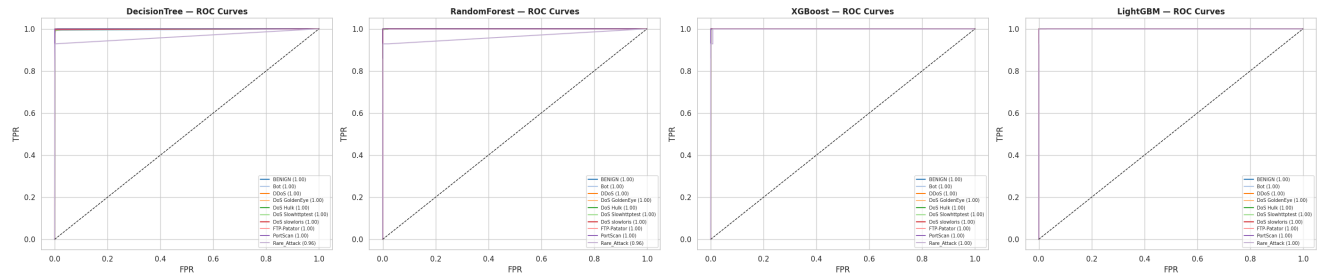
```
  F1 Macro           : 0.9214   ← quan trọng nhất (imbalanced)
  F1 Weighted        : 0.9959
  ROC-AUC (macro)    : 0.9997   ← độc lập threshold
  Balanced Accuracy  : 0.9138   ← accuracy công bằng mỗi class
  MCC                : 0.9952   ← [-1, 1], tin cậy nhất khi imbalanced
  Accuracy (ref only): 0.9961   ← không dùng để so sánh model
                        precision    recall  f1-score   support

                BENIGN       1.00      1.00      1.00     10000
                   Bot       0.99      0.99      0.99       390
                  DDoS       1.00      1.00      1.00     10000
        DoS GoldenEye       1.00      1.00      1.00      2057
             DoS Hulk       1.00      1.00      1.00     10000
     DoS Slowhttptest       1.00      0.99      0.99      1046
        DoS slowloris       0.99      0.99      0.99      1077
          FTP-Patator       1.00      1.00      1.00      1186
             PortScan       1.00      1.00      1.00     10000
           Rare_Attack       0.80      0.86      0.83        14
           SSH-Patator       1.00      1.00      1.00       644
Web Attack  Brute Force       0.75      0.71      0.73       294
     Web Attack  XSS       0.42      0.46      0.44       130

            accuracy                           1.00     46838
           macro avg       0.92      0.92      0.92     46838
        weighted avg       1.00      1.00      1.00     46838

  F1 Macro           : 0.9206   ← quan trọng nhất (imbalanced)
  F1 Weighted        : 0.9954
  ROC-AUC (macro)    : 0.9997   ← độc lập threshold
  Balanced Accuracy  : 0.9231   ← accuracy công bằng mỗi class
  MCC                : 0.9943   ← [-1, 1], tin cậy nhất khi imbalanced
  Accuracy (ref only): 0.9954   ← không dùng để so sánh model
```

```
# ── Confusion Matrices ──
fig, axes = plt.subplots(1, len(trained), figsize=(7 * len(trained), 6))
if len(trained) == 1: axes = [axes]

for ax, (name, res) in zip(axes, results.items()):
    cm = confusion_matrix(y_test, res["y_pred"])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=le.classes_, yticklabels=le.classes_,
                ax=ax, annot_kws={"size": 7})
    ax.set_title(f"{name}\nConfusion Matrix", fontweight="bold")
    ax.set_xlabel("Predicted"); ax.set_ylabel("True")
    ax.tick_params(axis="x", rotation=45, labelsize=7)
    ax.tick_params(axis="y", labelsize=7)

plt.tight_layout()
plt.savefig(f"{CONFIG['output_dir']}confusion_matrices.png", dpi=150, bbox_inches="tight")
plt.show()
```




```
# ── ROC Curves ──
fig, axes = plt.subplots(1, len(trained), figsize=(7 * len(trained), 6))
if len(trained) == 1: axes = [axes]

n_cls = len(le.classes_)
y_bin = label_binarize(y_test, classes=range(n_cls))
colors = plt.cm.tab20.colors

for ax, (name, res) in zip(axes, results.items()):
    y_prob = res["y_prob"]
```

```
    max_cls = min(n_cls, 10)
    for i in range(max_cls):
        if y_bin[:, i].sum() == 0: continue
        fpr, tpr, _ = roc_curve(y_bin[:, i], y_prob[:, i])
        auc_i = roc_auc_score(y_bin[:, i], y_prob[:, i])
        ax.plot(fpr, tpr, color=colors[i % 20],
                label=f"{le.classes_[i]} ({auc_i:.2f})", linewidth=1.5)
    ax.plot([0,1],[0,1],"k--", linewidth=1)
    ax.set_title(f"{name} — ROC Curves", fontweight="bold")
    ax.set_xlabel("FPR"); ax.set_ylabel("TPR")
    ax.legend(fontsize=7, loc="lower right")

plt.tight_layout()
plt.savefig(f"{CONFIG['output_dir']}roc_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```




```
df_cmp = pd.DataFrame([
    {"Model": name, "F1 Macro": r["f1_macro"],
     "F1 Weighted": r["f1_weighted"], "ROC-AUC (macro)": r["roc_auc"]}
    for name, r in results.items()
])
df_cmp = df_cmp.sort_values("F1 Macro", ascending=False).reset_index(drop=True)

# Highlight
def highlight_best(s):
    is_best = s == s.max()
    return ["background-color: #c8e6c9; font-weight: bold" if v else "" for v in is_best]

display(df_cmp.style
        .apply(highlight_best, subset=["F1 Macro","F1 Weighted","ROC-AUC (macro)"])
        .format({"F1 Macro": "{:.4f}", "F1 Weighted": "{:.4f}", "ROC-AUC (macro)": "{:.4f}"}))
```

	Model	F1 Macro	F1 Weighted	ROC-AUC (macro)
0	XGBoost	**0.9214**	**0.9959**	0.9997
1	LightGBM	0.9206	0.9954	**0.9997**
2	RandomForest	0.9202	0.9948	0.9969
3	DecisionTree	0.9092	0.9908	0.9869

Sau khi SMOTE

```
models = {
    "DecisionTree": DecisionTreeClassifier(
        max_depth=15,
        min_samples_split=5,
        min_samples_leaf=2,
        class_weight="balanced",
        random_state=42
    ),
    "RandomForest": RandomForestClassifier(
        n_estimators=200, max_depth=20, min_samples_split=5,
        class_weight="balanced", n_jobs=-1, random_state=42
    ),
    "XGBoost": XGBClassifier(
        n_estimators=200, max_depth=8, learning_rate=0.1,
```

```
        n_estimators=200, max_depth=6, learning_rate=0.1,
        subsample=0.8, colsample_bytree=0.8,
        eval_metric="mlogloss", n_jobs=-1, random_state=42,
        verbosity=0
    ),
    "LightGBM": LGBMClassifier(
        n_estimators=300, max_depth=8, learning_rate=0.05,
        num_leaves=63, subsample=0.8, colsample_bytree=0.8,
        class_weight="balanced", n_jobs=-1, random_state=42,
        verbose=-1
    ),
}

trained = {}
import time

for name, model in models.items():
    print(f"\n{'-'*50}")
    print(f"🏋️  Training {name}...")
    t0 = time.time()
    model.fit(X_train_res, y_train_res)
    elapsed = time.time() - t0
    trained[name] = model
    print(f"✅ Done in {elapsed:.1f}s")
```

```
--------------------------------------------------
🏋️  Training DecisionTree...
✅ Done in 12.1s

--------------------------------------------------
🏋️  Training RandomForest...
✅ Done in 124.7s

--------------------------------------------------
🏋️  Training XGBoost...
✅ Done in 156.9s

--------------------------------------------------
🏋️  Training LightGBM...
✅ Done in 212.1s
```

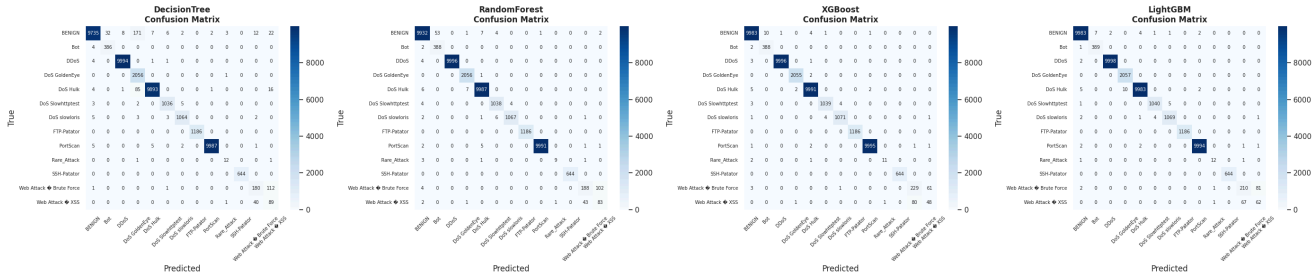
📊 10. Đánh giá Models

```
results = {}
for name, model in trained.items():
    results[name] = evaluate(model, name)
```

```
  Balanced Accuracy : 0.9163   ← accuracy cân bằng mỗi class
```

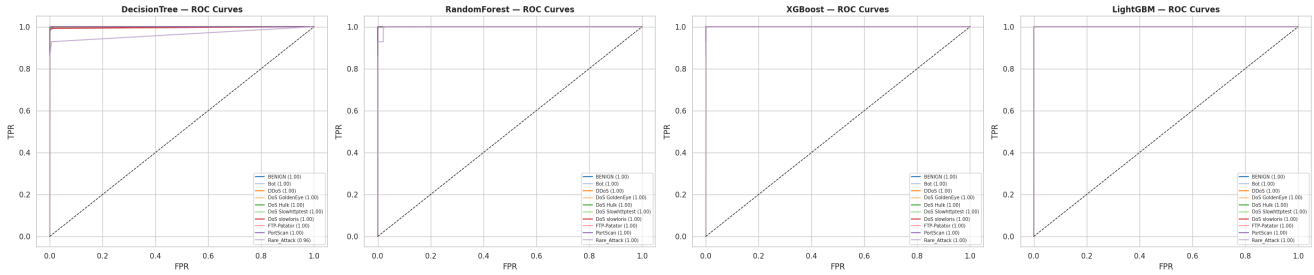
```
  Balanced Accuracy  : 0.9163   ← accuracy công bằng mỗi class
  MCC                : 0.9947   ← [-1, 1], tin cậy nhất khi imbalanced
  Accuracy (ref only): 0.9957   ← không dùng để so sánh model
                          precision    recall  f1-score   support

                  BENIGN       1.00      1.00      1.00     10000
                     Bot       0.98      1.00      0.99       390
                   DDoS       1.00      1.00      1.00     10000
          DoS GoldenEye       1.00      1.00      1.00      2057
               DoS Hulk       1.00      1.00      1.00     10000
       DoS Slowhttptest       1.00      0.99      0.99      1046
          DoS slowloris       0.99      0.99      0.99      1077
            FTP-Patator       1.00      1.00      1.00      1186
               PortScan       1.00      1.00      1.00     10000
            Rare_Attack       0.86      0.86      0.86        14
            SSH-Patator       1.00      1.00      1.00       644
Web Attack  Brute Force       0.75      0.71      0.73       294
        Web Attack  XSS       0.43      0.48      0.45       130

               accuracy                           1.00     46838
              macro avg       0.92      0.93      0.92     46838
           weighted avg       1.00      1.00      1.00     46838

  F1 Macro           : 0.9241   ← quan trọng nhất (imbalanced)
  F1 Weighted        : 0.9955
  ROC-AUC (macro)    : 0.9997   ← độc lập threshold
  Balanced Accuracy  : 0.9253   ← accuracy công bằng mỗi class
  MCC                : 0.9945   ← [-1, 1], tin cậy nhất khi imbalanced
  Accuracy (ref only): 0.9955   ← không dùng để so sánh model
```

```
# ── Confusion Matrices ──
fig, axes = plt.subplots(1, len(trained), figsize=(7 * len(trained), 6))
if len(trained) == 1: axes = [axes]

for ax, (name, res) in zip(axes, results.items()):
    cm = confusion_matrix(y_test, res["y_pred"])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=le.classes_, yticklabels=le.classes_,
                ax=ax, annot_kws={"size": 7})
    ax.set_title(f"{name}\nConfusion Matrix", fontweight="bold")
    ax.set_xlabel("Predicted"); ax.set_ylabel("True")
    ax.tick_params(axis="x", rotation=45, labelsize=7)
    ax.tick_params(axis="y", labelsize=7)

plt.tight_layout()
plt.savefig(f"{CONFIG['output_dir']}confusion_matrices.png", dpi=150, bbox_inches="tight")
plt.show()
```

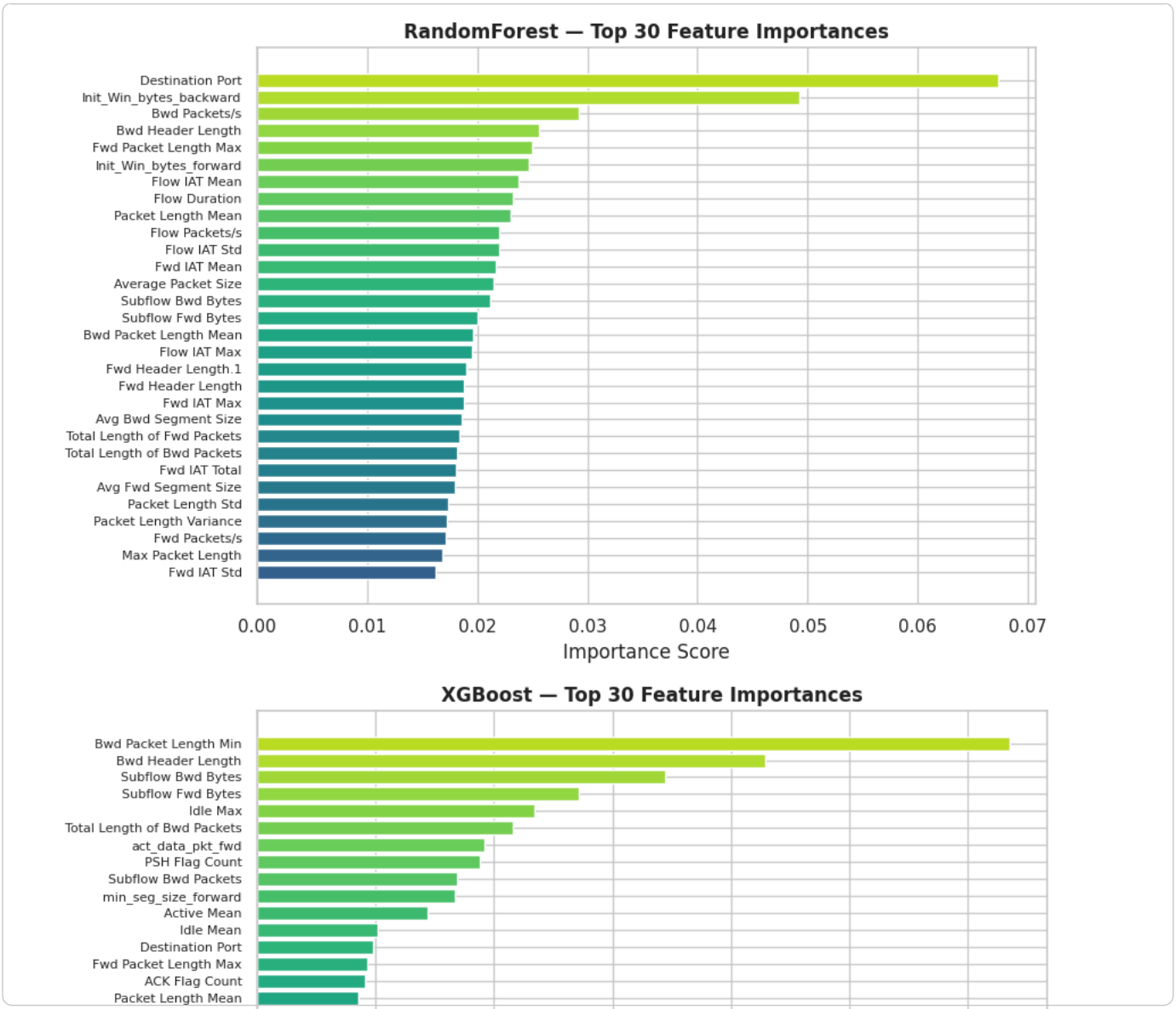



```
# ── ROC Curves ──
fig, axes = plt.subplots(1, len(trained), figsize=(7 * len(trained), 6))
if len(trained) == 1: axes = [axes]

n_cls = len(le.classes_)
y_bin = label_binarize(y_test, classes=range(n_cls))
colors = plt.cm.tab20.colors

for ax, (name, res) in zip(axes, results.items()):
    y_prob = res["y_prob"]
    max_cls = min(n_cls, 10)
    for i in range(max_cls):
        if y_bin[:, i].sum() == 0: continue
        fpr, tpr, _ = roc_curve(y_bin[:, i], y_prob[:, i])
```

```
        auc_i = roc_auc_score(y_bin[:, i], y_prob[:, i])
        ax.plot(fpr, tpr, color=colors[i % 20],
                label=f"{le.classes_[i]} ({auc_i:.2f})", linewidth=1.5)
    ax.plot([0,1],[0,1],"k--", linewidth=1)
    ax.set_title(f"{name} — ROC Curves", fontweight="bold")
    ax.set_xlabel("FPR"); ax.set_ylabel("TPR")
    ax.legend(fontsize=7, loc="lower right")

plt.tight_layout()
plt.savefig(f"{CONFIG['output_dir']}roc_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```




```
# —— Feature Importance (Random Forest & XGBoost) ——
top_n = 30
for name in ["RandomForest", "XGBoost", "DecisionTree", "LightGBM"]:
    if name not in trained: continue
    model = trained[name]
    imp = model.feature_importances_
    top_idx = np.argsort(imp)[::-1][:top_n]
    top_names = [X.columns[i] for i in top_idx]
    top_vals  = imp[top_idx]

    fig, ax = plt.subplots(figsize=(9, 6))
    bars = ax.barh(range(top_n), top_vals[::-1], color=plt.cm.viridis(np.linspace(0.3, 0.9, top_n)))
    ax.set_yticks(range(top_n))
    ax.set_yticklabels(top_names[::-1], fontsize=8)
    ax.set_xlabel("Importance Score")
    ax.set_title(f"{name} — Top {top_n} Feature Importances", fontweight="bold")
    plt.tight_layout()
    plt.savefig(f"{CONFIG['output_dir']}{name}_feature_importance.png", dpi=150, bbox_inches="tight")
    plt.show()
```



🏆 11. So sánh Models

```
df_cmp = pd.DataFrame([
    {"Model": name, "F1 Macro": r["f1_macro"],
     "F1 Weighted": r["f1_weighted"], "ROC-AUC (macro)": r["roc_auc"]}
    for name, r in results.items()
])
df_cmp = df_cmp.sort_values("F1 Macro", ascending=False).reset_index(drop=True)

# Highlight
def highlight_best(s):
    is_best = s == s.max()
    return ["background-color: #c8e6c9; font-weight: bold" if v else "" for v in is_best]

display(df_cmp.style
        .apply(highlight_best, subset=["F1 Macro","F1 Weighted","ROC-AUC (macro)"])
        .format({"F1 Macro": "{:.4f}", "F1 Weighted": "{:.4f}", "ROC-AUC (macro)": "{:.4f}"}))
```

	Model	F1 Macro	F1 Weighted	ROC-AUC (macro)
0	LightGBM	**0.9241**	0.9955	0.9997
1	XGBoost	0.9209	**0.9956**	**0.9997**
2	RandomForest	0.9149	0.9943	0.9996
3	DecisionTree	0.9068	0.9883	0.9869

```python
# ── CELL 10.5: Demo dự đoán 30 mẫu — TẤT CẢ MODEL ──

print("=" * 70)
print("  DEMO: Dự đoán 30 mẫu ngẫu nhiên từ Test Set — Tất cả Model")
print("=" * 70)

np.random.seed(99)

# Lấy 30 mẫu — mỗi class lấy khoảng 2-3 mẫu
sample_indices = []
n_per_class = max(1, 30 // len(le.classes_))  # ~2 mẫu/class với 13 classes
for cls_i in range(len(le.classes_)):
    idx = np.where(y_test == cls_i)[0]
    if len(idx) == 0:
        continue
    n_take = min(n_per_class, len(idx))
    sample_indices.extend(np.random.choice(idx, size=n_take, replace=False))

# Nếu chưa đủ 30, lấy thêm ngẫu nhiên từ các class lớn
if len(sample_indices) < 30:
    remaining = list(set(range(len(y_test))) - set(sample_indices))
    extra = np.random.choice(remaining, size=30-len(sample_indices), replace=False)
    sample_indices.extend(extra)

sample_indices = sample_indices[:30]
np.random.shuffle(sample_indices)  # xáo trộn để không theo thứ tự class

X_sample   = X_test_s[sample_indices]
y_true_idx = y_test[sample_indices]
true_labels = [le.classes_[i] for i in y_true_idx]

# ── Bảng so sánh tất cả model ──
print("\n📋 Kết quả dự đoán từng model:\n")

comparison_rows = []
for sample_i, true_label in enumerate(true_labels):
    row = {"#": f"#{sample_i+1}", "True Label": true_label}
    for model_name, model in trained.items():
        pred_idx   = model.predict(X_sample[sample_i:sample_i+1])[0]
        pred_prob  = model.predict_proba(X_sample[sample_i:sample_i+1])[0]
        pred_label = le.classes_[pred_idx]
        confidence = pred_prob[pred_idx] * 100
        correct    = "✅" if pred_label == true_label else "❌"
        row[model_name] = f"{correct} {pred_label} ({confidence:.0f}%)"
    comparison_rows.append(row)

df_compare = pd.DataFrame(comparison_rows)

# Print dạng text
print(df_compare.to_string(index=False))

# Styled display
def highlight_cell(val):
    if "✅" in str(val): return "background-color: #c8e6c9; text-align: center"
    elif "❌" in str(val): return "background-color: #ffcdd2; text-align: center"
    return "text-align: center"

display(df_compare.style
        .applymap(highlight_cell)
        .set_caption("So sánh dự đoán — 30 mẫu Test × 4 Models")
        .set_properties(**{"text-align": "center", "font-size": "10px"}))

# ── Accuracy trên 30 mẫu demo ──
print("\n📊 Accuracy trên 30 mẫu demo:")
print("—" * 45)
for model_name, model in trained.items():
    preds   = le.classes_[model.predict(X_sample)]
    correct = sum(p == t for p, t in zip(preds, true_labels))
    bar     = "▓" * correct + "⬚" * (30 - correct)
    print(f"  {model_name:20s}: {correct:2d}/30  ({correct/30*100:.0f}%)  {bar}")

# ── Heatmap confidence: 30 mẫu × n_classes cho từng model ──
fig, axes = plt.subplots(len(trained), 1,
                         figsize=(22, len(trained) * 5))

for row_i, (model_name, model) in enumerate(trained.items()):
    ax = axes[row_i]
```

```
    y_prob_matrix = model.predict_proba(X_sample)  # (30, n_classes)
    y_pred_all    = model.predict(X_sample)

    im = ax.imshow(y_prob_matrix.T, aspect="auto",
                   cmap="YlOrRd", vmin=0, vmax=1)

    # Trục x: 30 mẫu
    ax.set_xticks(range(30))
    xlabels = []
    for i, (true_i, pred_i) in enumerate(zip(y_true_idx, y_pred_all)):
        mark = "✅" if true_i == pred_i else "❌"
        xlabels.append(f"#{i+1}\n{mark}")
    ax.set_xticklabels(xlabels, fontsize=6.5)

    # Trục y: class names
    ax.set_yticks(range(len(le.classes_)))
    ax.set_yticklabels(le.classes_, fontsize=7)

    # Đánh dấu true class và predicted class
    for sample_i, (true_i, pred_i) in enumerate(zip(y_true_idx, y_pred_all)):
        # True class → viền xanh
        ax.add_patch(plt.Rectangle(
            (sample_i - 0.5, true_i - 0.5), 1, 1,
            fill=False, edgecolor="green", linewidth=2
        ))
        # Sai → viền đỏ ở predicted
        if pred_i != true_i:
            ax.add_patch(plt.Rectangle(
                (sample_i - 0.5, pred_i - 0.5), 1, 1,
                fill=False, edgecolor="red", linewidth=2, linestyle="--"
            ))

    ax.set_title(f"{model_name} — Confidence Heatmap (30 mẫu)",
                 fontweight="bold", fontsize=10)
    ax.set_xlabel("Sample", fontsize=8)
    ax.set_ylabel("Class", fontsize=8)
    plt.colorbar(im, ax=ax, fraction=0.02, pad=0.01, label="Confidence")

plt.suptitle("Confidence Heatmap — 30 mẫu Test × Tất cả Model\n"
             "🟩 Viền xanh = True class  |  🟥 Viền đỏ đứt = Predicted sai",
             fontweight="bold", fontsize=12)
plt.tight_layout()
plt.savefig(f"{CONFIG['output_dir']}demo_30samples_heatmap.png",
            dpi=150, bbox_inches="tight")
plt.show()

# ── Consensus ──
print("\n📦  Consensus — 30 mẫu:")
print("─" * 75)
n_all_correct = 0
n_majority    = 0
n_all_wrong   = 0

for sample_i, true_label in enumerate(true_labels):
    preds       = [le.classes_[m.predict(X_sample[sample_i:sample_i+1])[0]]
                   for m in trained.values()]
    n_correct   = preds.count(true_label)
    all_correct = n_correct == len(trained)
    consensus   = "✅ Tất cả đúng" if n_correct == len(trained) else \
                  "🟡 Đa số đúng" if n_correct >= len(trained) // 2 + 1 else \
                  "❌ Đa số sai"

    if n_correct == len(trained):   n_all_correct += 1
    elif n_correct >= len(trained) // 2 + 1: n_majority += 1
    else:                           n_all_wrong   += 1

    print(f"  #{sample_i+1:2d} [{true_label:28s}] {consensus}  "
          f"({n_correct}/{len(trained)} đúng)  "
          f"Votes: {' | '.join(preds)}")

print(f"\n📊 Tổng kết consensus 30 mẫu:")
print(f"  ✅ Tất cả model đúng : {n_all_correct:2d}/30 ({n_all_correct/30*100:.0f}%)")
print(f"  🟡 Đa số đúng        : {n_majority:2d}/30 ({n_majority/30*100:.0f}%)")
print(f"  ❌ Đa số sai         : {n_all_wrong:2d}/30 ({n_all_wrong/30*100:.0f}%)")
```

```
======================================================================
  DEMO: Dự đoán 30 mẫu ngẫu nhiên từ Test Set — Tất cả Model
======================================================================

📋 Kết quả dự đoán từng model:

  #              True Label                    DecisionTree                    RandomForest
 #1              Rare_Attack                ✅ Rare_Attack (100%)           ✅ Rare_Attack (99%)          ✅ Rare
 #2              FTP-Patator                ✅ FTP-Patator (100%)           ✅ FTP-Patator (100%)         ✅ FTP-
 #3         Web Attack  XSS              ❌ Web Attack  Brute Force (100%) ❌ Web Attack  Brute Force (58%) ❌ Web Attack  Bru
 #4              SSH-Patator                ✅ SSH-Patator (100%)           ✅ SSH-Patator (100%)         ✅ SSH-
 #5         DoS Slowhttptest           ✅ DoS Slowhttptest (100%)      ✅ DoS Slowhttptest (100%)      ✅ DoS Slowl
 #6                      Bot                   ✅ Bot (100%)                   ✅ Bot (95%)
 #7         DoS Slowhttptest           ✅ DoS Slowhttptest (100%)      ✅ DoS Slowhttptest (100%)      ✅ DoS Slowl
 #8                 PortScan                ✅ PortScan (100%)              ✅ PortScan (100%)            ✅ P
 #9                   BENIGN                ✅ BENIGN (90%)                 ✅ BENIGN (98%)               ✅
#10            DoS GoldenEye            ✅ DoS GoldenEye (100%)         ✅ DoS GoldenEye (100%)         ✅ DoS Go
#11                 PortScan                ✅ PortScan (100%)              ✅ PortScan (100%)            ✅ P
#12                   BENIGN                ✅ BENIGN (100%)                ✅ BENIGN (100%)              ✅
#13 Web Attack  Brute Force            ❌ Web Attack  XSS (62%)        ❌ Web Attack  XSS (53%)  ✅ Web Attack  Bru
#14            DoS slowloris            ✅ DoS slowloris (100%)         ✅ DoS slowloris (100%)         ✅ DoS s
#15                    DDoS                   ✅ DDoS (100%)                  ✅ DDoS (100%)
#16                    DDoS                   ✅ DDoS (100%)                  ✅ DDoS (100%)
#17                 DoS Hulk                ✅ DoS Hulk (100%)              ✅ DoS Hulk (100%)            ✅ D
#18            DoS GoldenEye            ✅ DoS GoldenEye (100%)         ✅ DoS GoldenEye (100%)         ✅ DoS Go
#19                      Bot                   ✅ Bot (99%)                    ✅ Bot (99%)
#20                 DoS Hulk                ✅ DoS Hulk (100%)              ✅ DoS Hulk (100%)            ✅ D
#21 Web Attack  Brute Force            ❌ Web Attack  XSS (62%)        ❌ Web Attack  XSS (73%)  ✅ Web Attack  Bru
#22                 PortScan                ✅ PortScan (100%)              ✅ PortScan (100%)            ✅ P
#23              Rare_Attack            ❌ Web Attack  XSS (90%)        ❌ BENIGN (52%)               ✅ Ra
#24                 DoS Hulk                ✅ DoS Hulk (100%)              ✅ DoS Hulk (100%)            ✅ D
#25            DoS slowloris            ✅ DoS slowloris (100%)         ✅ DoS slowloris (100%)         ✅ DoS s
#26                    DDoS                   ✅ DDoS (100%)                  ✅ DDoS (100%)
#27              SSH-Patator                ✅ SSH-Patator (100%)           ✅ SSH-Patator (100%)         ✅ SSH-
#28                   BENIGN                ✅ BENIGN (100%)                ✅ BENIGN (100%)              ✅
#29              FTP-Patator                ✅ FTP-Patator (100%)           ✅ FTP-Patator (100%)         ✅ FTP-
#30          Web Attack  XSS              ✅ Web Attack  XSS (62%)  ❌ Web Attack  Brute Force (52%) ❌ Web Attack  Bru
```

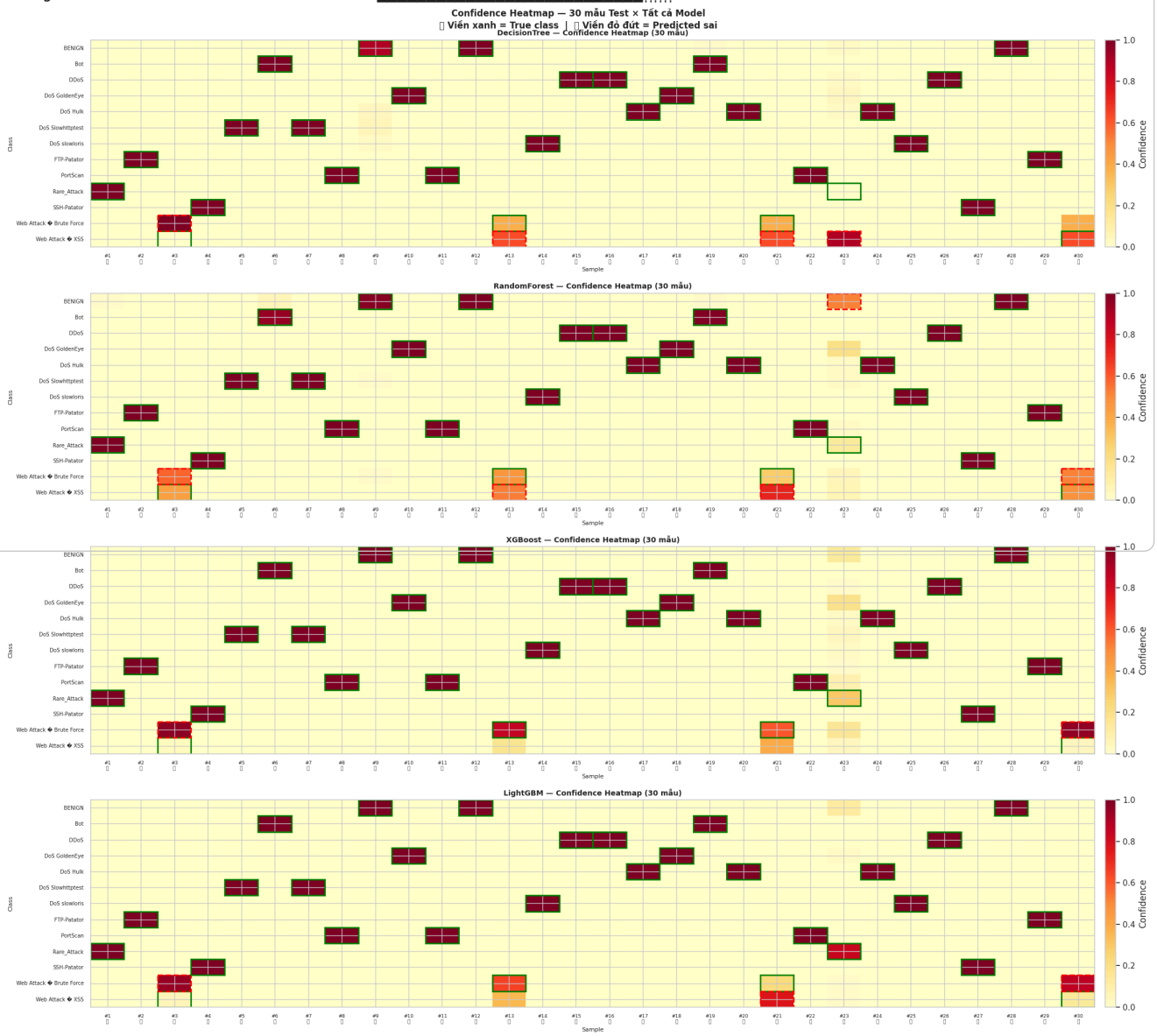
So sánh dự đoán — 30 mẫu Test × 4 Models

	#	True Label	DecisionTree	RandomForest	XGBoost	LightGBM
0	#1	Rare_Attack	✅ Rare_Attack (100%)	✅ Rare_Attack (99%)	✅ Rare_Attack (100%)	✅ Rare_Attack (100%)
1	#2	FTP-Patator	✅ FTP-Patator (100%)	✅ FTP-Patator (100%)	✅ FTP-Patator (100%)	✅ FTP-Patator (100%)
2	#3	Web Attack XSS	❌ Web Attack Brute Force (100%)	❌ Web Attack Brute Force (58%)	❌ Web Attack Brute Force (97%)	❌ Web Attack Brute Force (95%)
3	#4	SSH-Patator	✅ SSH-Patator (100%)	✅ SSH-Patator (100%)	✅ SSH-Patator (100%)	✅ SSH-Patator (100%)
4	#5	DoS Slowhttptest	✅ DoS Slowhttptest (100%)	✅ DoS Slowhttptest (100%)	✅ DoS Slowhttptest (100%)	✅ DoS Slowhttptest (100%)
5	#6	Bot	✅ Bot (100%)	✅ Bot (95%)	✅ Bot (100%)	✅ Bot (100%)
6	#7	DoS Slowhttptest	✅ DoS Slowhttptest (100%)	✅ DoS Slowhttptest (100%)	✅ DoS Slowhttptest (100%)	✅ DoS Slowhttptest (100%)
7	#8	PortScan	✅ PortScan (100%)	✅ PortScan (100%)	✅ PortScan (100%)	✅ PortScan (100%)
8	#9	BENIGN	✅ BENIGN (90%)	✅ BENIGN (98%)	✅ BENIGN (100%)	✅ BENIGN (100%)
9	#10	DoS GoldenEye	✅ DoS GoldenEye (100%)	✅ DoS GoldenEye (100%)	✅ DoS GoldenEye (100%)	✅ DoS GoldenEye (100%)
10	#11	PortScan	✅ PortScan (100%)	✅ PortScan (100%)	✅ PortScan (100%)	✅ PortScan (100%)
11	#12	BENIGN	✅ BENIGN (100%)	✅ BENIGN (100%)	✅ BENIGN (100%)	✅ BENIGN (100%)
12	#13	Web Attack Brute Force	❌ Web Attack XSS (62%)	❌ Web Attack XSS (53%)	✅ Web Attack Brute Force (84%)	✅ Web Attack Brute Force (65%)
13	#14	DoS slowloris	✅ DoS slowloris (100%)	✅ DoS slowloris (100%)	✅ DoS slowloris (100%)	✅ DoS slowloris (100%)
14	#15	DDoS	✅ DDoS (100%)	✅ DDoS (100%)	✅ DDoS (100%)	✅ DDoS (100%)
15	#16	DDoS	✅ DDoS (100%)	✅ DDoS (100%)	✅ DDoS (100%)	✅ DDoS (100%)
16	#17	DoS Hulk	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)
17	#18	DoS GoldenEye	✅ DoS GoldenEye (100%)	✅ DoS GoldenEye (100%)	✅ DoS GoldenEye (100%)	✅ DoS GoldenEye (100%)
18	#19	Bot	✅ Bot (99%)	✅ Bot (99%)	✅ Bot (100%)	✅ Bot (100%)
19	#20	DoS Hulk	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)
20	#21	Web Attack Brute Force	❌ Web Attack XSS (62%)	❌ Web Attack XSS (73%)	✅ Web Attack Brute Force (60%)	❌ Web Attack XSS (77%)
21	#22	PortScan	✅ PortScan (100%)	✅ PortScan (100%)	✅ PortScan (100%)	✅ PortScan (100%)
22	#23	Rare_Attack	❌ Web Attack XSS (90%)	❌ BENIGN (52%)	✅ Rare_Attack (30%)	✅ Rare_Attack (83%)
23	#24	DoS Hulk	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)	✅ DoS Hulk (100%)
24	#25	DoS slowloris	✅ DoS slowloris (100%)	✅ DoS slowloris (100%)	✅ DoS slowloris (100%)	✅ DoS slowloris (100%)

25	#26	DDoS	✅ DDoS (100%)	✅ DDoS (100%)	✅ DDoS (100%)	✅ DDoS (100%)
26	#27	SSH-Patator	✅ SSH-Patator (100%)	✅ SSH-Patator (100%)	✅ SSH-Patator (100%)	✅ SSH-Patator (100%)
27	#28	BENIGN	✅ BENIGN (100%)	✅ BENIGN (100%)	✅ BENIGN (100%)	✅ BENIGN (100%)
28	#29	FTP-Patator	✅ FTP-Patator (100%)	✅ FTP-Patator (100%)	✅ FTP-Patator (100%)	✅ FTP-Patator (100%)
29	#30	Web Attack ▯ XSS	✅ Web Attack ▯ XSS (62%)	❌ Web Attack ▯ Brute Force (52%)	❌ Web Attack ▯ Brute Force (96%)	❌ Web Attack ▯ Brute Force (86%)

📊 Accuracy trên 30 mẫu demo:

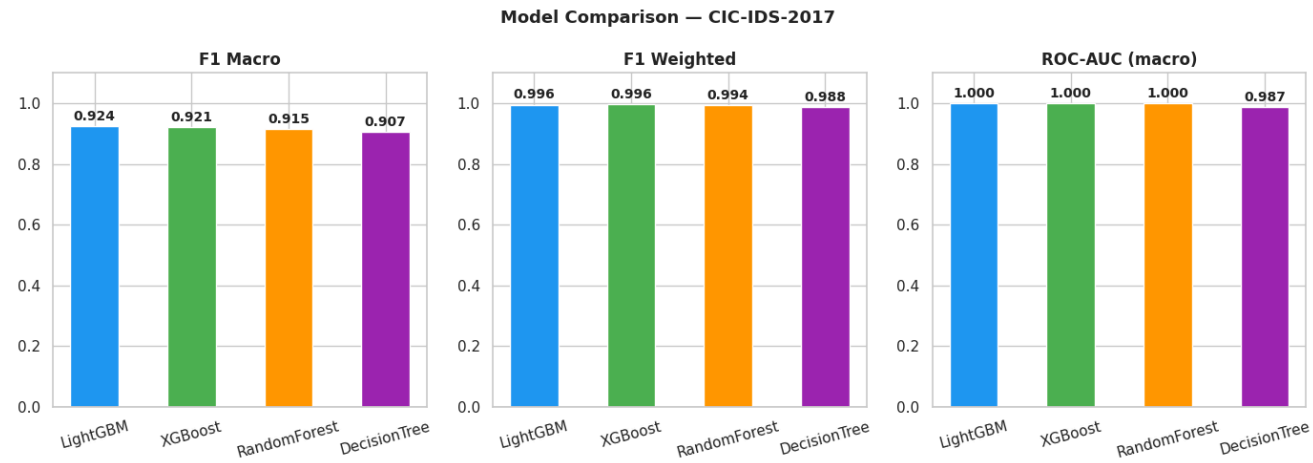
```
DecisionTree        : 26/30  (87%)
RandomForest        : 25/30  (83%)
XGBoost             : 28/30  (93%)
LightGBM            : 27/30  (90%)
```



📦 Consensus — 30 mẫu:

```
# 1 [Rare_Attack              ] ✅ Tất cả đúng  (4/4 đúng)  Votes: Rare_Attack | Rare_Attack | Rare_Attack | Rar
# 2 [FTP-Patator              ] ✅ Tất cả đúng  (4/4 đúng)  Votes: FTP-Patator | FTP-Patator | FTP-Patator | FTP
# 3 [Web Attack ▯ XSS         ] ❌ Đa số sai  (0/4 đúng)  Votes: Web Attack ▯ Brute Force | Web Attack ▯ Brute F
# 4 [SSH-Patator              ] ✅ Tất cả đúng  (4/4 đúng)  Votes: SSH-Patator | SSH-Patator | SSH-Patator | SSH
# 5 [DoS Slowhttptest         ] ✅ Tất cả đúng  (4/4 đúng)  Votes: DoS Slowhttptest | DoS Slowhttptest | DoS Slo
# 6 [Bot                      ] ✅ Tất cả đúng  (4/4 đúng)  Votes: Bot | Bot | Bot | Bot
# 7 [DoS Slowhttptest         ] ✅ Tất cả đúng  (4/4 đúng)  Votes: DoS Slowhttptest | DoS Slowhttptest | DoS Slo
# 8 [PortScan                 ] ✅ Tất cả đúng  (4/4 đúng)  Votes: PortScan | PortScan | PortScan | PortScan
# 9 [BENIGN                   ] ✅ Tất cả đúng  (4/4 đúng)  Votes: BENIGN | BENIGN | BENIGN | BENIGN
#10 [DoS GoldenEye            ] ✅ Tất cả đúng  (4/4 đúng)  Votes: DoS GoldenEye | DoS GoldenEye | DoS GoldenEye
#11 [PortScan                 ] ✅ Tất cả đúng  (4/4 đúng)  Votes: PortScan | PortScan | PortScan | PortScan
#12 [BENIGN                   ] ✅ Tất cả đúng  (4/4 đúng)  Votes: BENIGN | BENIGN | BENIGN | BENIGN
#13 [Web Attack ▯ Brute Force ] ❌ Đa số sai  (2/4 đúng)  Votes: Web Attack ▯ XSS | Web Attack ▯ XSS | Web Attac
#14 [DoS slowloris            ] ✅ Tất cả đúng  (4/4 đúng)  Votes: DoS slowloris | DoS slowloris | DoS slowloris
```

```
# Bar chart so sánh
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
metrics = ["F1 Macro", "F1 Weighted", "ROC-AUC (macro)"]
palette = ["#2196F3", "#4CAF50", "#FF9800", "#9C27B0"]

for ax, metric in zip(axes, metrics):
    vals = df_cmp[metric].fillna(0).values
    bars = ax.bar(df_cmp["Model"], vals, color=palette[:len(df_cmp)], edgecolor="white", width=0.5)
    ax.set_ylim(0, 1.1)
    ax.set_title(metric, fontweight="bold")
    ax.tick_params(axis="x", rotation=15)
    for bar, v in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2, v + 0.01, f"{v:.3f}",
                ha="center", va="bottom", fontsize=10, fontweight="bold")

plt.suptitle("Model Comparison — CIC-IDS-2017", fontweight="bold", fontsize=13)
plt.tight_layout()
plt.savefig(f"{CONFIG['output_dir']}model_comparison.png", dpi=150, bbox_inches="tight")
plt.show()

df_cmp.to_csv(f"{CONFIG['output_dir']}model_comparison.csv", index=False)
print("\n✅ Kết quả đã lưu vào /content/ids_results/")
```




```
✅ Kết quả đã lưu vào /content/ids_results/
```

```
# ================================================================
# CELL 1: Save models + artifacts lên Google Drive / local
# ================================================================
import pickle, os, json
import numpy as np
from google.colab import drive

SAVE_DIR = "/content/ids_results/IDS_Models/"
os.makedirs(SAVE_DIR, exist_ok=True)

# Save từng model
for name, model in trained.items():
    path = os.path.join(SAVE_DIR, f"{name}.pkl")
    with open(path, "wb") as f:
        pickle.dump(model, f)
    size_mb = os.path.getsize(path) / 1e6
    print(f"✅ Saved {name:20s} → {path}  ({size_mb:.1f} MB)")

# Save scaler + label encoder (bắt buộc phải có khi deploy)
with open(os.path.join(SAVE_DIR, "scaler.pkl"), "wb") as f:
    pickle.dump(scaler, f)

with open(os.path.join(SAVE_DIR, "label_encoder.pkl"), "wb") as f:
```

```
    pickle.dump(le, f)

# Save feature names (để đảm bảo thứ tự cột đúng khi predict)
feature_names = list(X.columns)
with open(os.path.join(SAVE_DIR, "feature_names.json"), "w") as f:
    json.dump(feature_names, f, indent=2)

# Save metadata
metadata = {
    "classes":       list(le.classes_),
    "n_features":    len(feature_names),
    "feature_names": feature_names,
    "models":        list(trained.keys()),
    "best_model":    df_cmp.iloc[0]["Model"],   # model có F1 Macro cao nhất
}
with open(os.path.join(SAVE_DIR, "metadata.json"), "w") as f:
    json.dump(metadata, f, indent=2)

print(f"\n📁 Tất cả đã lưu vào: {SAVE_DIR}")
print(f"🏆 Best model: {metadata['best_model']}")
```

📩 12. Download kết quả

```
from google.colab import files
import zipfile, os

# Zip toàn bộ kết quả
zip_path = "/content/ids_results.zip"
with zipfile.ZipFile(zip_path, "w") as zf:
    for f in os.listdir(CONFIG["output_dir"]):
        zf.write(os.path.join(CONFIG["output_dir"], f), f)

print("📦 Các file trong zip:")
with zipfile.ZipFile(zip_path) as zf:
    for name in zf.namelist():
        print(f"  - {name}")

files.download(zip_path)
print("\n✅ Download hoàn tất!")
```

📝 Ghi chú & Giải thích

Tại sao dùng Combined Sampling?

Vấn đề	Giải pháp
BENIGN chiếm ~80% → model bias	RandomUnderSampler cắt xuống 3× class lớn thứ 2
Một số attack < 100 mẫu	SMOTE tạo synthetic samples
Class cực hiếm (< 500)	Gộp vào `Rare_Attack`

Metric nào quan trọng nhất?

- **F1 Macro**: Đánh giá đồng đều tất cả class (quan trọng khi class imbalance)
- **ROC-AUC**: Đo khả năng phân biệt từng loại attack
- Trong IDS thực tế: **Recall (Sensitivity)** của attack class quan trọng hơn Precision (thà false alarm hơn bỏ sót attack)

📋 Tổng kết & Đánh giá Pipeline — CIC-IDS-2017

---

1. Tổng quan thí nghiệm

Hạng mục	Chi tiết
Dataset	CIC-IDS-2017 — Canadian Institute for Cybersecurity
Raw data	2,830,743 rows × 79 columns
Sau làm sạch	2,520,798 rows (xóa 5,734 Inf/NaN + 307,078 duplicate)
Sau sampling	234,187 rows (tối đa 50,000 mẫu/class)
Train / Test	187,349 / 46,838 (80/20, stratified)

Hạng mục	Chi tiết
Số classes	13 (10 attack types + BENIGN + Rare_Attack)
Số features	78 numeric features

---

2. Xử lý Class Imbalance

Dataset gốc có mức độ mất cân bằng rất nghiêm trọng ($IR = 2{,}273{,}097/11 \approx 206{,}645$). Pipeline áp dụng chiến lược **Combined Sampling**:

Bước 1 — Stratified Sampling: Giới hạn mỗi class tối đa 50,000 mẫu để tránh OOM. 3 class cực hiếm (Infiltration: 36, SQL Injection: 21, Heartbleed: 11) được gộp vào nhãn `Rare_Attack`.

Bước 2 — Proportional SMOTE (công thức Square Root Target):

$$N_{\text{target}} = \min\left(\sqrt{N_{\text{maj}} \times N_{\min}},\ 10 \times N_{\min},\ N_{\text{maj}}\right)$$

Thay vì oversample tất cả class lên cùng mức (gây thổi phồng), target được tính tỉ lệ với số mẫu thực — tránh tình trạng class chỉ có 54 mẫu bị synthetic lên 40,000 (tỷ lệ synthetic > 99%).

Class	$N_{\text{thực}}$	Target	% Synthetic	Độ tin cậy
DoS GoldenEye	8,229	18,142	~55%	🟡
FTP-Patator	4,745	13,776	~66%	🟡
Bot	1,558	7,894	~80%	⚠️
Web Attack - XSS	522	4,569	~89%	🔴
Rare_Attack	54	540	~90%	🔴

---

3. Kết quả so sánh Models

3.1 Trước SMOTE (chỉ dùng `class_weight="balanced"`)

Model	F1 Macro	F1 Weighted	ROC-AUC	Train time
XGBoost	**0.9214**	**0.9959**	0.9997	112s
LightGBM	0.9206	0.9954	**0.9997**	149s
RandomForest	0.9202	0.9948	0.9969	62s
DecisionTree	0.9092	0.9908	0.9869	7s

3.2 Sau SMOTE (Proportional SMOTE)

Model	F1 Macro	F1 Weighted	ROC-AUC	Balanced Acc	MCC	Train time
LightGBM	**0.9241**	0.9955	**0.9997**	**0.9253**	0.9945	212s
XGBoost	0.9209	**0.9956**	**0.9997**	0.9163	**0.9947**	157s
RandomForest	0.9149	0.9943	0.9996	—	—	125s
DecisionTree	0.9068	0.9883	0.9869	—	—	12s

3.3 Tác động của SMOTE lên từng model (ΔF1 Macro)

Model	Trước	Sau	Thay đổi
LightGBM	0.9206	0.9241	**+0.0035** ✅
XGBoost	0.9214	0.9209	-0.0005 ≈ 0
RandomForest	0.9202	0.9149	-0.0053 ⚠️
DecisionTree	0.9092	0.9068	-0.0024 ⚠️

> **Nhận xét:** SMOTE cải thiện rõ rệt LightGBM nhờ histogram-based splitting tận dụng tốt synthetic samples. Ngược lại, RF và DT đã có `class_weight="balanced"` nên SMOTE thêm vào chỉ tạo noise nhẹ. XGBoost ổn định nhất — ít bị ảnh hưởng bởi resampling.

---

4. Phân tích per-class (Model tốt nhất: LightGBM sau SMOTE)

4.1 Class mạnh — F1 ≥ 0.99

Class	Precision	Recall	F1	Lý do
BENIGN	1.00	1.00	1.00	Đủ data, pattern đặc trưng
DDoS	1.00	1.00	1.00	Volume cao, dễ nhận dạng
DoS Hulk	1.00	1.00	1.00	Pattern HTTP flood rõ ràng
PortScan	1.00	1.00	1.00	Sequential port access đặc trưng
FTP-Patator	1.00	1.00	1.00	Brute force pattern rõ
SSH-Patator	1.00	1.00	1.00	Brute force pattern rõ